JAAFR INTERNATIONAL RESEARCH JOURNAL
JOURNAL OF ADVANCE AND FUTURE RESEARCH
JAAFR.ORG | ISSN : 2984-889X
An International Open Access, Peer-reviewed, Refereed Journal

IDPS:IOT NETWORK TRAFFIC ANALYZER AND THREAT DETECTION DASHBOARD

A Machine Learning-Based IoT Network Traffic Analysis and Threat Detection

[1]Adish Rajesh , [2]Faheem Hakeem, [3]Mohammed Daneen , [4]Mohammed Fahad , [5]Ms.Chithrakshi

[1,2,3,4]Undergraduate Students, [5]Assistant Professor

[12345]Department of CSE (Artificial Intelligence and Machine Learning),

[12345]Shree Devi Institute of Technology,Mangaluru, Karnataka India -574142

Abstract : IoT networks bring together devices with different capabilities, protocols, and communication patterns. Because these devices continuously exchange data, manually checking the traffic is not a practical way to find every suspicious event. In this work, we present IDPS, an IoT Network Traffic Analyzer and Threat Detection Dashboard developed to monitor traffic, extract useful features, classify suspicious activity, estimate its risk, and apply a response policy. The prototype combines a FastAPI backend, React dashboard, Scapy packet processing, a Random Forest classifier, SQLite storage, and an ESP32 test device. The project training data contains 15,000 structured samples organised according to the CICIoT2023 attack taxonomy and represented by nine traffic features. Training uses a stratified 75/25 split with a 120-tree Random Forest. The project documentation reports 99.42% accuracy, 99.26% precision, 99.53% recall, and 99.39% F1-score. When the packaged data was independently rerun using the documented hold-out procedure, all four measures reached 100%. We report the two results separately because they come from different runs. The system also converts the model output and behavioural information into a 0–100 risk score. Depending on that score, an event may be monitored, temporarily quarantined, or permanently blocked. The prototype demonstrates the complete detection-to-response workflow, while its evaluation also points to the need for larger physical tests and independent datasets.

IndexTerms - Internet of Things, intrusion detection and prevention, Random Forest, machine learning, network traffic analysis, CICIoT2023, ESP32, risk scoring, FastAPI, React.

I. INTRODUCTION

The number of connected sensors, controllers, gateways, and embedded devices has grown rapidly, and this growth has increased the possible attack surface of IoT environments [1]. IoT devices also differ from conventional enterprise endpoints in several practical ways. They may have limited processing power, use different communication protocols, remain in service for long periods, and provide only a small amount of local logging [2]. In such situations, network

traffic becomes a useful source of evidence because suspicious behaviour can sometimes be recognised even when detailed information from the device is unavailable.

Traditional rule-based monitoring is still useful for known attack patterns, but fixed rules may not work as well when normal traffic changes or when an attack appears in a new form [3]. Machine-learning methods provide another option by learning relationships between measurable traffic characteristics, including packet volume, request frequency, connection behaviour, and failed connections [4]. Even then, a classifier does not solve the whole operational problem. Someone managing the network still needs to know how serious an event is and what should happen after it has been detected.

IDPS was developed with this wider requirement in mind. The prototype connects traffic collection, feature extraction, Random Forest classification, risk scoring, policy enforcement, storage, and dashboard monitoring in one workflow. An ESP32 is used for the physical part of the demonstration, while controlled simulation provides another way to repeat the experiments without requiring many physical IoT devices.

1.1 Research Contributions

This work makes six practical contributions. First, it provides an end-to-end architecture that links IoT traffic capture with a web dashboard. Second, it uses nine selected traffic features as a compact representation for classification. Third, a Random Forest model is used to identify suspicious events. Fourth, the model output is combined with behavioural and historical information to form a 0–100 risk score. Fifth, the system includes temporary-quarantine and permanent-blocking policies. Finally, the design combines an ESP32 test setup with simulation so that the proposed workflow can be demonstrated under controlled conditions.

1.2 Problem Statement

The problem considered in this work is not simply detecting suspicious IoT traffic. The system must continue to analyse traffic from different devices and provide a response that an administrator can understand and act on. Raw network activity contains many individual events, so the useful information has to be reduced to a manageable set of features without making inference unnecessarily expensive. Alerts should also be separated according to their severity rather than treating every unusual event as equally dangerous. IDPS addresses these requirements through a sequence of traffic processing, classification, risk assessment, and response steps.

II. NEED FOR THE STUDY

IoT devices do not all communicate at the same rate or use the same services. For example, a sudden increase in packet volume, repeated connection attempts, an unusual request rate, or a large number of failed connections may indicate scanning, denial-of-service activity, brute-force attempts, spoofing, or another form of unwanted behaviour [8]. As the number of devices increases, checking these changes manually becomes difficult and time-consuming.

There is another issue after detection: deciding what to do. A dashboard that only raises an alert still requires the administrator to judge its seriousness and take the next step. In IDPS, the risk engine and policy layer are used for this purpose. Low-risk events can remain under observation, whereas events that reach the configured high or critical levels can be restricted automatically.

III. OBJECTIVES

• Develop a real-time dashboard for monitoring IoT network activity and identifying possible threats.
• Capture or simulate IoT traffic and convert the resulting events into a consistent feature set.
• Train and integrate a Random Forest model for detecting suspicious traffic.
• Produce a readable risk score from the model result and related behavioural indicators.
• Apply temporary quarantine or permanent blocking when the configured severity calls for it.
• Demonstrate the system with an ESP32 while keeping simulation available for repeatable tests.
• Keep device, alert, and policy records for dashboard analysis and later auditing.

IV. RELATED WORK

IoT intrusion-detection research has covered several approaches, including supervised learning, unsupervised methods, deep learning, and network-behaviour analysis [4], [5]. Chaabouni et al. [2] reviewed learning-based approaches and discussed the difficulty of dealing with the variety of IoT devices and traffic. Meidan et al. [6] showed that botnet activity can be identified from network traffic using autoencoders. Antonakakis et al. [7] examined the Mirai botnet and showed how compromised IoT devices can become part of larger attacks. CICIoT2023 [8] provides a benchmark containing a broad range of IoT attack categories.

These studies provide a strong reason for using machine learning in traffic analysis. However, a working monitoring system has additional concerns. It needs features that can actually be collected, a model that responds quickly enough, a clear policy for handling alerts, and a dashboard that gives the operator useful information. The aim of IDPS is therefore to connect these elements instead of evaluating the classifier separately from the rest of the system.

4.1 Research Gap

One difference between many experimental IDS studies and a system intended for practical use is the treatment of the response stage. Classification metrics can show how well a model identifies events, but they do not tell the operator what action should follow. In the proposed system, the model result is combined with behavioural information and passed to a risk engine. The resulting score is then used by the policy layer to decide the appropriate response.

V. PROPOSED SYSTEM

The proposed architecture is divided into six functional areas: the IoT devices and traffic sources, traffic acquisition and feature extraction, machine-learning inference, risk and policy processing, backend storage, and dashboard presentation. FastAPI coordinates the backend services. Scapy is used when packet-level capture is available, while application-level events can be used for simulation or for environments in which direct packet access is restricted.

5.1 IoT Device and ESP32 Layer

An ESP32 acts as the physical IoT node in the demonstration. Its firmware connects to the network over Wi-Fi and can send normal telemetry or generate controlled traffic. Another controlled scenario can be used to produce abnormal communication, giving the backend a measurable change to analyse. The ESP32 does not run the classifier; the classification step is handled by the backend.

5.2 Backend and Dashboard

The backend is written in Python with FastAPI. SQLAlchemy and Pydantic are used for application data handling. On the client side, the dashboard uses React and TypeScript with Vite and Tailwind CSS. Axios, Recharts, and WebSocket communication support the display of device status, traffic information, alerts, risk levels, and policy actions. SQLite is used as the default database, although the design can be moved to PostgreSQL if required.

5.3 Real-Time Monitoring

When an event reaches the backend, the required features are calculated first. The trained model then produces a prediction, after which the risk state is updated and the event is stored. The latest information is sent to the dashboard through the real-time communication layer. In this way, the demonstration covers the complete path from an incoming event to a visible security decision.

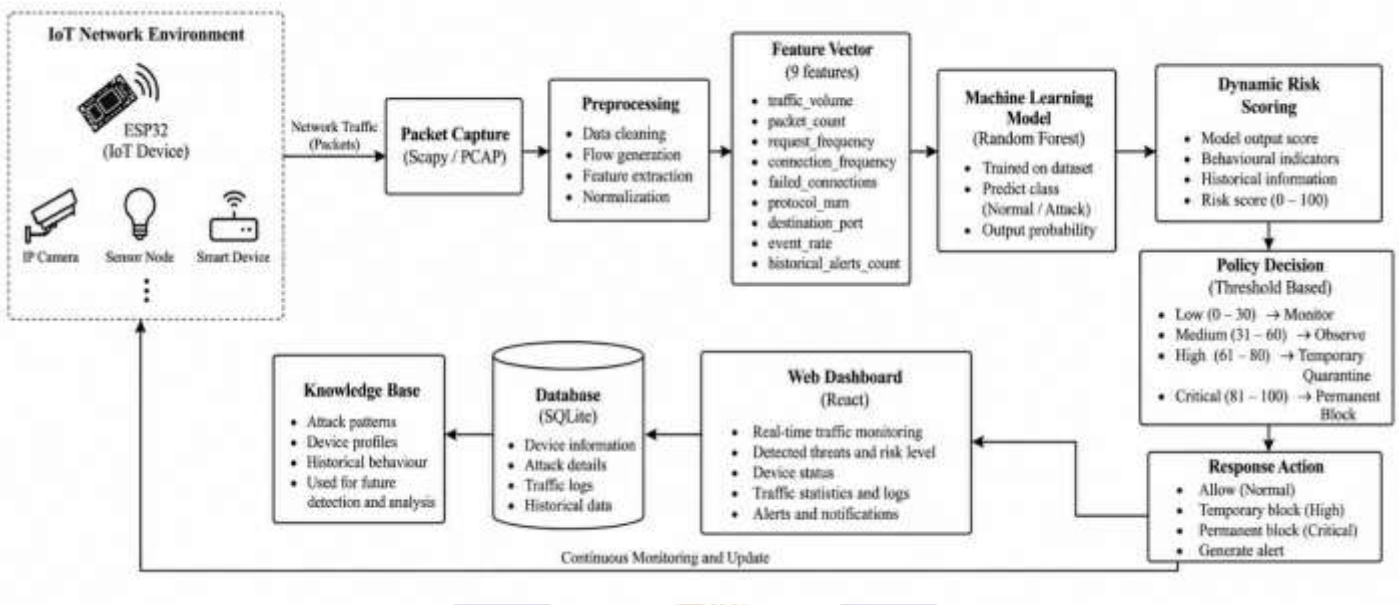


Figure 1. System workflow of the proposed IoT network traffic analyzer and threat detection dashboard(IDPS).

VI. DATASET AND FEATURE REPRESENTATION

6.1 CICIoT2023 Dataset Context

CICIoT2023 is a benchmark created for studying large-scale attacks in IoT environments [8]. It contains 33 attacks grouped into seven categories: DDoS, DoS, Recon, Web-based attacks, brute force, spoofing, and Mirai. The original benchmark includes traffic captures and extracted data from a diverse IoT environment. In this project, its attack taxonomy is used to organise the structured training data rather than claiming that the project dataset is the complete original release. Ring et al. [9] provide a broader survey of network intrusion-detection datasets.

6.2 Project Training Dataset

The project dataset contains 15,000 structured records. Of these, 8,250 are benign and 6,750 are suspicious. The suspicious records are distributed as follows: DDoS 2,015, DoS 1,333, Recon 1,226, Mirai 973, BruteForce 558, Web 366, and Spoofing 279. For this reason, the dataset is more accurately described as project training data aligned with the CICIoT2023 taxonomy, rather than as the complete raw CICIoT2023 dataset.

6.3 Nine Standardized Features

The classifier receives nine flow-level features: traffic_volume, packet_count, request_frequency, connection_frequency, failed_connections, protocol_num, destination_port, event_rate, and historical_alerts_count. Together, they give the model information about traffic intensity, communication frequency, failed activity, protocol and port details, and earlier alerts. The compact representation also means that the model does not need to process the complete packet payload.

6.4 Feature Importance

The project's feature-importance analysis gives the highest value to event_rate, at approximately 0.2922. The next two are request_frequency at 0.2338 and connection_frequency at 0.2264. Packet_count has an importance of about 0.1420, while traffic_volume is about 0.0735. The remaining features have smaller contributions, with protocol_num and destination_port close to zero in the fitted model.

VII. MACHINE LEARNING METHODOLOGY

7.1 Model Selection

Random Forest was chosen for the classifier because the selected traffic variables can have non-linear relationships, while the model remains practical for the dashboard application [10]. The configuration uses 120 trees, a maximum depth of 14, a minimum split size of 4, and a minimum leaf size of 2. Balanced class weights are enabled and the random state is fixed at 42.

7.2 Training Procedure

A stratified 75/25 split is used to separate the data into training and testing portions, with random state 42. This gives 11,250 training samples and 3,750 test samples. The feature names and their order are kept the same when predictions are made later by the application. After training, the model is saved with Joblib so the FastAPI service can load it during startup instead of retraining it every time.

7.3 Inference Workflow

For an incoming event, the feature extractor prepares the nine values required by the classifier. Random Forest then returns the suspiciousness prediction. That result is forwarded to the risk engine, where it is considered along with the other behavioural information. Keeping capture, feature preparation, inference, and response as separate stages makes the implementation easier to follow and test.

VIII. DYNAMIC RISK SCORING AND AUTOMATED RESPONSE

A binary prediction alone does not provide much information about the degree of risk. The IDPS risk engine therefore produces a score from 0 to 100. The calculation uses the machine-learning result together with traffic volume, request frequency, connection frequency, failed connections, domain-related risk, and previous alert information. This score is then used to determine how the event should appear and what response it should receive.

8.1 Risk Levels and Actions

Four ranges are used for the risk score: 0–30 is Low, 31–60 is Medium, 61–80 is High, and 81–100 is Critical. With the current policy, High-risk events are temporarily quarantined and Critical events are permanently blocked. A temporary restriction has an expiry time, and a background worker checks for restrictions that have reached their expiry.

8.2 Mitigation Scope

The present implementation carries out blocking at the application or simulation level. It does not directly modify the operating system's firewall rules. That approach is adequate for demonstrating the project, but it should not be treated as a complete production enforcement mechanism. A real deployment would need secure authentication, stronger access controls, audit logging, and carefully controlled integration with firewall or network-access systems.

IX. EXPERIMENTAL EVALUATION

9.1 Classification Results

According to the project documentation, the classifier achieved 99.42% accuracy, 99.26% precision, 99.53% recall, and 99.39% F1-score. We also repeated the documented experiment using the packaged 15,000-row dataset, the stated Random Forest configuration, and the stratified 75/25 split. The resulting 3,750-sample hold-out set gave 100% for accuracy, precision, recall, and F1-score. These results are presented separately because the first set is the documented project result and the second is the result of our own reproduction.

9.2 Latency

The project documentation reports approximately 1.42 ms for Random Forest inference, 27.02 ms for average end-to-end mitigation, and 42.40 ms at the 95th percentile. These measurements belong to the particular development setup used for the project. They should therefore not be read as fixed performance values, since hardware, operating-system configuration, traffic volume, and deployment conditions can change the observed timings.

9.3 Software Validation

There are 12 test functions in the repository covering areas such as authentication, blocking, device events, Random Forest behaviour, and simulation. We were not able to complete the full test run in the current environment because the test setup stopped when the bcrypt dependency was missing. The tests therefore show the areas that were intended to be validated, but they do not establish that every test passes under every environment.

X. DISCUSSION, LIMITATIONS AND FUTURE WORK

The evaluation indicates that the nine selected features are sufficient to support fast inference in the project's development environment and fit naturally into the dashboard workflow. The very high hold-out scores still need to be interpreted with care. The packaged dataset has a fixed structure and contains 15,000 samples, so strong results on this dataset alone cannot show how the model will perform on unfamiliar devices, networks, protocols, or attack variants.

There are several limitations in the current implementation. The training data has a limited scope, the classifier depends on the selected nine features, blocking is performed at the application layer, and the physical ESP32 setup is relatively small. A production system would additionally need stronger identity and access controls, secure communication,

detailed audit records, ongoing model monitoring, and protection against attempts to manipulate the traffic features used by the detector.

The next stage of the work can involve testing with independent IoT datasets and extending the binary detector to identify individual attack classes. MQTT- and CoAP-specific features would also be useful because IoT deployments often rely on these protocols. Other possible improvements include studying changes in traffic over time, creating separate behavioural baselines for devices, and providing clearer explanations for alerts. Connecting the response mechanism to a network firewall or software-defined networking controller would make the mitigation stage closer to a real deployment. A larger ESP32 testbed with controlled attack scenarios would also provide a stronger basis for evaluation.

XI. CONCLUSION

The work presented here combines traffic monitoring, nine-feature extraction, Random Forest classification, risk scoring, and automated response in one IoT security prototype. The system uses an ESP32 physical device together with controlled simulation, while FastAPI handles the backend and React provides the dashboard. The resulting workflow allows suspicious events to move from traffic collection through classification and risk assessment to a configured response.

The experimental results suggest that the selected feature representation and Random Forest setup can provide quick inference in the development environment. At the same time, the difference between documented and reproduced results shows why transparent reporting matters. The present implementation should therefore be considered a prototype and a useful base for further development, rather than a finished production-grade intrusion prevention system.

ACKNOWLEDGMENT

We would like to thank our guide, Ms. Chithrakshi, Assistant Professor, for her guidance and encouragement during the development of the IoT Network Traffic Analyzer and Threat Detection Dashboard project.

We also thank the faculty members of the Department of Computer Science and Engineering at Shree Devi Institute of Technology, Mangaluru, for their support and the institution for providing the resources and infrastructure required for the work. We are grateful to our parents, friends, and well-wishers for their continued support during the project.

REFERENCES

[1] S. M. Tahsien, H. Karimipour, and P. Spachos, “Machine learning based solutions for security of Internet of Things (IoT): A survey,” J. Netw. Comput. Appl., vol. 161, Art. no. 102630, 2020.

[2] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, “Network intrusion detection for IoT security based on learning techniques,” IEEE Commun. Surveys Tuts., vol. 21, no. 3, pp. 2671–2701, 2019.

[3] A. A. Ghorbani, W. Lu, and M. Tavallaee, Network Intrusion Detection and Prevention: Concepts and Techniques. Boston, MA, USA: Springer, 2010.

[4] K. A. P. da Costa, J. P. Papa, C. O. Lisboa, R. Munoz, and V. H. C. de Albuquerque, “Internet of Things: A survey on machine learning-based intrusion detection approaches,” Comput. Netw., vol. 151, pp. 147–157, 2019.

[5] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, “Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study,” J. Inf. Secur. Appl., vol. 50, Art. no. 102419, 2020.

[6] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, "N-BaIoT—Network-based detection of IoT botnet attacks using deep autoencoders," IEEE Pervasive Comput., vol. 17, no. 3, pp. 12–22, Jul. 2018, doi: 10.1109/MPRV.2018.03367731.

[7] M. Antonakakis et al., "Understanding the Mirai botnet," in Proc. 26th USENIX Security Symp., Vancouver, BC, Canada, 2017, pp. 1093–1110.

[8] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," Sensors, vol. 23, no. 13, Art. no. 5941, 2023.

[9] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," Comput. Secur., vol. 86, pp. 147–167, 2019.

[10] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.